New York University — Tandon School of Engineering
Department of Technology Management and Innovation
IE-GY 9113B — Systems Integration: From ERP to Agentic AI

Study Note — Week 7: From RPA to Intelligent Automation

Module 3: Intelligent Automation + AI Agents | Friday, October 16, 2026

Readings for this week

Primary case	JPMorgan **COiN** (Contract Intelligence) — the machine-learning system JPMorgan built to read commercial loan agreements that had taken 360,000 lawyer- and loan-officer-hours a year. Journalism of record (subscription): Son, Bloomberg (2017) — bloomberg.com/news/articles/2017-02-28 Open access: ABA Journal, Weiss (2017) — abajournal.com/news/article (jpmorgan_chase_uses_tech…)
Go deeper (optional)	Iansiti and Lakhani, *Competing in the Age of AI*, Ch. 3 (The AI Factory); Tella, Brinker and Pezzini, *The New Automation Mindset*, Ch. 15 (The Enterprise AI Platform); Bornet et al., *Agentic Artificial Intelligence*, Ch. 1–2. *Reference, not required reading — the note teaches the ideas directly and points you to the source at the exact place it uses each one.*
Before next session	Read the Week 8 study note and Bornet et al. Ch. 1–2 before the Week 8 session — they are the on-ramp to the SPAR agent framework you will build in Lab 3.

You Broke the Bot at the Judgment Step — This Week You Meet What Takes It

This course climbs through four stages — integrate, automate, make intelligent, make agentic — and each rung is built on the one beneath it. In Module 1 you built the first: the integration backbone, the connected set of systems through which a company's orders, plans, and cash actually move. In Module 2 you built the second: an automation layer on that backbone, using robotic process automation to take over the repetitive, rules-based work — and in Lab 2 you produced a working bot on your own process, wired two systems together, and then deliberately broke it. Those two modules are the ground this one stands on. Module 3 opens here, and it is the third stage: make intelligent — and, in the weeks that follow, make agentic. You stop asking a machine only to follow a rule you wrote and begin asking it to read a situation and exercise judgment, and then to pursue a goal on its own. The move is from RPA, to intelligent automation, to AI agents — up the ladder, building on what you already have rather than replacing it.

Hold on to the exact place Lab 2 left you, because this whole week grows out of it. On your own process map you marked two kinds of place where a decision gets made. The first was a stable-rules decision point: a step where the rule covers every case, where the same inputs always produce the same correct output. That was your RPA candidate — repetitive, rules-based, stable — and in Lab 2 you built a bot for it and watched it run. The second was a judgment decision point: a step where the rule runs out, where a real case does not fit the template and a person has to weigh it. In Lab 2 you fed your bot the rejection and rework rows you had seeded, and it did exactly what a rule-follower does — it applied its rule with perfect confidence to a case the rule did not fit, and got it wrong without pausing. That failure was not a bug. It was the boundary of rules-based automation, drawn in your own data. This week names what lives on the other side of that boundary, and Lab 3 will have you build it.

Keep the three threads in your ear, because Module 3 re-weights them. Through Module 2 the lead thread was Operations and Supply Chain, because that is where repetitive, rules-based work is generated and where a bot programme finds most of its targets. Module 3 moves the Customer thread to the front, because intelligent and agentic systems are deployed first at customer-facing surfaces — service, fulfilment, dispute resolution — where the return is most visible and the governance stakes are highest. That is where the arc is heading. This week's case, though, lands first on Operations and Cash Management, because the cleanest early example of a machine taking a judgment step is a back-office one: a bank reading its own loan contracts. Watch the thread move as the module unfolds — the mechanism is proved in the back office this week, and carried to the customer front over the weeks that follow.

And keep the four archetypes in view, because the judgment step you will design for looks different at each scale: the consumer-packaged-goods manufacturer running sales-and-operations planning on SAP; the mid-market manufacturer running order-to-cash on NetSuite; the Series-B startup stitching order-to-cash across Stripe, Shopify and QuickBooks; and the fifty-person services firm living in HubSpot and Notion. A large bank's contract-reading engine and a small firm's one ambiguous approval are the same move at different sizes — a machine taking a step that used to need a person's judgment — and the governing idea holds at every scale.

One map will organise the whole module, so meet it now. Bornet and his co-authors borrow the way the Society of Automotive Engineers grades self-driving cars — six levels from fully manual to fully autonomous — and apply the same idea to automation (Bornet et al., 2025). The levels run from Level 0, where humans do everything, through Level 1, rule-based automation, to Level 5, a theoretical fully autonomous agent. You do not need the whole ladder today; you need to locate yourself on it. Everything you built in Module 2 lived at Level 1 — basic, rules-driven, no judgment. This week is the step up to Level 2, where automation gains cognitive abilities and begins to handle work a rule cannot. The agent arc of the coming weeks lives at Level 3 and above. The ladder is the spine of Module 3; the rest of the note is about taking the first step up it.

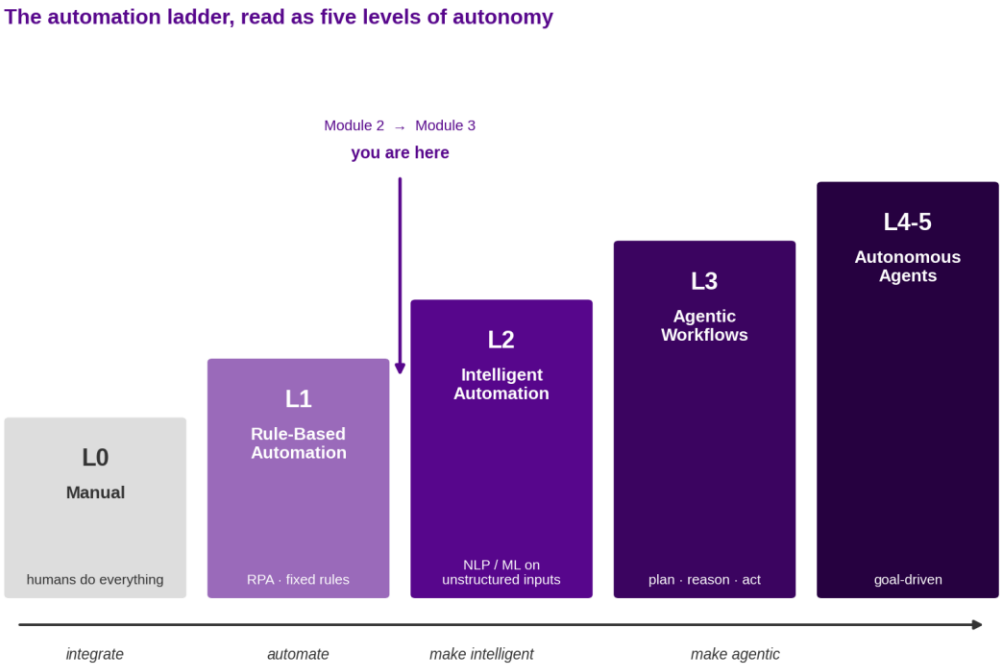


Figure 1. The course arc read as levels of autonomy. Module 2 lived at Level 1 (rule-based automation); this week is the step to Level 2 (intelligent automation). The agent arc of the coming weeks lives at Level 3 and above. Adapted from Bornet et al. (2025), Ch. 2.

The boundary you drew in Lab 2 is this week's subject. You marked two kinds of decision point on your own map: the stable-rules point — your RPA candidate, built in Lab 2 — and the judgment point, where the rule ran out and the bot failed. This week is about what takes that second step. You are not leaving rule-based automation behind: you are adding a rung above it — intelligent automation, Level 2 — for the steps a rule cannot hold, on top of the integration backbone and the RPA layer you already built. And the Customer thread now leads the module, even though this week's case lands first on Operations and Cash.

The Week's Question: When the Rule Runs Out, What Takes the Decision — and What Changes When a Machine Does?

A rule-based bot is a marvel exactly where the world is tidy. Point it at a step whose logic is complete — every case decided, no "it depends" — and it will run that step faster and more reliably than a person, forever. But most real processes are not tidy all the way through. Somewhere in almost every flow there is a step where the inputs are messy, the cases vary, and the right answer requires reading a situation rather than matching a pattern. That is the judgment step, and it is where your bot broke. So the week turns on a two-part question, and both parts matter. First, the mechanical one: what kind of technology can take a step a rule cannot — reading an unstructured input, interpreting an ambiguous case, deciding what a situation calls for? Second, and harder, the governance one: when a machine takes a step that used to require expert human judgment, rather than repetitive clerk work, what changes about accountability — who is responsible, what must be transparent, and where does a human have to stay?

The first part is the easier half and the course answers it quickly. The second part is the one that will occupy the rest of Module 3 and your Board Memo, because it does not have a technical answer. Automating a clerk's keystrokes and automating a lawyer's reading of a contract are different in kind, not just in degree, and the difference is the whole subject of the week.

A Rule Runs Out Where Judgment Begins

Start with why rules-based automation hits a wall, because the wall is not a flaw in the tool — it is built into what a rule is. Recall the definition you have carried since Week 4: a software robot operates your existing applications through their interface, following a fixed script. It reads a field, checks it against a rule, and takes a pre-decided action. For that to work, two things must be true. The input has to be structured — sitting in a known field, in a known format, so the bot can find it — and the logic has to be complete — every possible case mapped to a decided action, with nothing left to weigh. Where both hold, you have a stable-rules decision point and a candidate for a bot. Where either fails, the rule runs out.

It fails in two ways, and you have met both. The first is messy input. A rule can compare a number to a threshold, but it cannot read a paragraph, interpret a photograph, or make sense of an email written in a customer's own words — because none of those arrive as a value in a field. The second is an incomplete rule — a case the logic did not anticipate. This is the one that bit you in Lab 2. The bot does not know that its rule has stopped fitting; it has no doubt, no pause, no sense that something is off. It applies the rule with full confidence to a case the rule was never written for and produces a confidently wrong result — and, worse, it does so silently, so the error can run for weeks before anyone notices. A human at that step would have caught it out of the corner of their eye. The bot cannot, because catching it is not following a rule; it is exercising judgment.

So a process map really has two kinds of decision point, and telling them apart is the core skill of the week. A control point is where the business commits or data is created. A decision point is where a routing choice is made — this case goes down this path, that case down another. Some decision points are stable: the rule decides every case, cleanly and forever. Those are RPA candidates, and you have already automated one. Other decision points require judgment: the cases vary, the rule cannot cover them all, and someone has to weigh the particular situation. Those are the steps a rule cannot hold — and they are exactly the steps an intelligent system is built to take. The costly pattern in a real process is almost always a judgment point mishandled as if it were a stable one: a case is pushed down the wrong path, sent back, worked by hand, and re-enters the flow only after someone has spent real effort on it. Rejection and rework are expensive precisely because they sit at judgment points that were automated as though they were not.

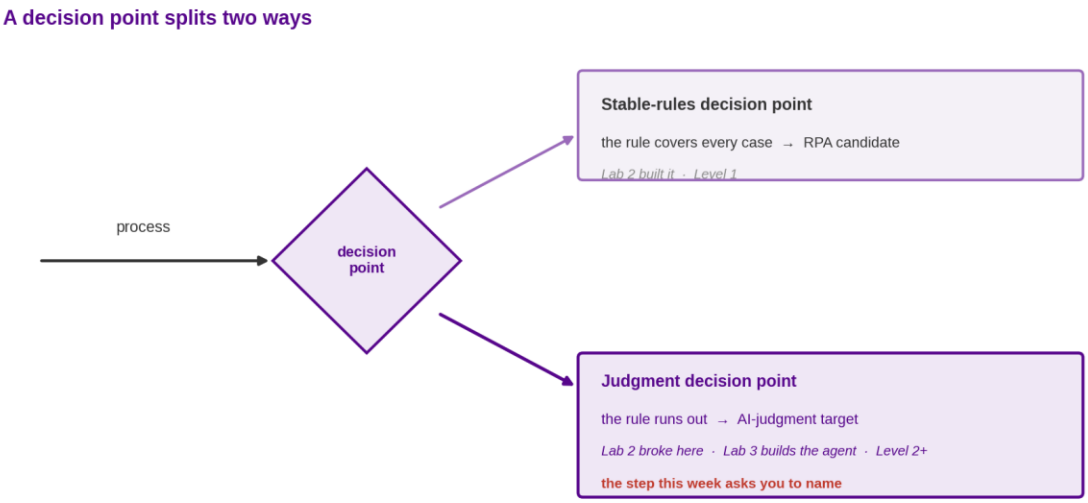


Figure 2. Every decision point splits two ways. Where the rule covers every case, it is a stable-rules point and an RPA candidate — the kind you automated in Lab 2. Where the rule runs out, it is a judgment point — the step this week asks you to name, and the target your agent will take in Lab 3.

Why does this boundary matter strategically, and not just technically? Because it tells you where more automation of the Level 1 kind stops adding value and starts adding risk. Pile more rules onto a judgment step and you do not get judgment; you get a longer, more brittle rulebook that still fails on the next case it did not foresee — and now the failure is harder to see and harder to fix. A company that does not know where its judgment points are will keep trying to automate them with rules, and will keep getting what Module 2 warned about: faster versions of the same dysfunction. Knowing exactly where the rule runs out is what lets you aim the next, more expensive layer of technology — intelligent automation — at the few steps that actually need it, and leave the cheap, reliable rule in place everywhere else. That discipline is the whole point of this week's homework.

What the step requires	Can a rule decide it?	What it is
Structured input, logic complete — every case mapped to a decided action	Yes — same input, same correct output, every time	Stable-rules decision point → RPA candidate (Level 1)
A case the rule did not anticipate — the template does not fit	No — the rule applies confidently and is wrong, silently	Judgment decision point → AI-judgment target (Level 2+)
Unstructured input — a paragraph, an email, an image, a contract clause	No — the value is not in a field the rule can read	Judgment decision point → needs NLP / ML to interpret

A rule needs a tidy world; judgment is for the untidy part. Rules-based automation works only where the input is structured and the logic is complete. It fails on messy inputs and on cases the rule did not foresee — and it fails silently, with full confidence. The step where the rule runs out is the judgment decision point. Naming it precisely is what lets you aim intelligent automation at the few steps that need it, rather than piling more rules onto a step that will never be ruled.

Teaching a Machine to Read What a Rule Cannot

If the limit of a rule is that it cannot read messy input or weigh an unforeseen case, then the technology that passes the limit must be able to do exactly those things. That technology is machine learning, and the shift it enables has a name: intelligent automation — rules-based automation augmented with cognitive abilities so it can interpret information and make decisions, not just execute steps (Bornet et al., 2025). Before the name, though, the idea, because the idea is what you have to be able to explain to a board.

Begin with the two words that do the work: unstructured data, and machine learning. Structured data sits in labelled fields — an amount, a date, a status — where a rule can find it. Unstructured data is everything else: the free text of an email, the clauses of a contract, the pixels of a scanned form, the audio of a support call. It carries meaning, but not in a place a rule can point at. Machine learning is what lets a system find the meaning anyway. Rather than being told the rule in advance, a machine-learning model is shown many examples and learns the pattern that connects an input to an outcome — the way, as Bornet and his co-authors put it, you teach a child what a dog is not with a definition but by pointing at dogs until the pattern forms (Bornet et al., 2025). The model does not follow a rule you wrote; it builds its own, from data, and applies it to inputs it has never seen.

Iansiti and Lakhani, in their chapter on what they call the AI factory, give you the working vocabulary for the three ways a machine learns, and you should be able to name each with an example (Iansiti and Lakhani, 2020). Supervised learning trains on examples that have already been labelled by a human expert — thousands of contracts a lawyer has already marked up, say — and learns to reproduce the expert's judgment on new, unlabelled cases. Unsupervised learning is given no labels; it is asked to find the natural groupings and the odd-one-out in the data on its own — which is why it is the engine behind anomaly detection, the flagging of the transaction or the record that does not fit the pattern. Reinforcement learning starts from a goal and a measure of success and learns by trying, keeping what improves its position and discarding what does not. You do not need the mathematics. You need to recognise which kind of problem each solves, because the judgment step on your own map will point you to one of them: reproducing an expert's reading of a document is supervised; catching the case that does not belong is unsupervised.

Now put the pieces together, because this is the move the whole week describes. Take a rule-based automation — the bot that follows a fixed script — and give it a machine-learning model to handle the part a rule cannot. The bot still does the structured, repetitive work; the model reads

the unstructured input or weighs the ambiguous case and hands back a decision; and the two run as one flow. Bornet and his co-authors give the clean example: an automated workflow reads a customer's email in their own words using natural-language processing, an AI classifier decides what the request is actually about, and a rule-based process then handles it (Bornet et al., 2025). The automation is no longer blind to meaning. The step your bot could not take — reading the message, interpreting the case — is now taken by a model, and only the structured remainder is left to the rule. That is intelligent automation, and the bank case at the centre of this week is its purest form.

Intelligent automation is a rule-based flow with a model where the judgment used to fail. Machine learning lets a system interpret unstructured input — text, images, contracts — and weigh cases a rule never anticipated, by learning patterns from examples rather than following a rule you wrote. The three families are supervised (reproduce a labelled expert judgment), unsupervised (find the anomaly), and reinforcement (learn by trying toward a goal). Intelligent automation bolts a model onto a rule-based flow so the model takes the judgment step and the rule keeps the structured remainder.

The Value Changes Too: From Cost Saved to Insight Gained

It would be easy to treat intelligent automation as merely a better bot — a machine that automates a slightly harder step and saves a little more money. That reading misses the real shift, and a board that misses it will fund the wrong thing. Rules-based automation sells on cost: it takes a repetitive task a person was doing and does it cheaper and faster, and the business case is a labour-hours calculation. Intelligent automation can do that too, but its distinctive value is different in kind. When a machine reads every contract, every transaction, every customer message — not a sample, but all of it — it does not just do the work more cheaply; it sees patterns no one was looking at, because no one had the hours to look. The value is not only the cost removed. It is the insight generated.

Iansiti and Lakhani give this its sharpest framing. The firms they study have built what they call an AI factory: a system that treats decision-making itself as an industrial process. Where the Industrial Revolution industrialised the making of goods, these firms industrialise the making of decisions — data pipelines gather and clean information, algorithms turn it into predictions, an experimentation platform tests which predictions actually work, and the whole thing runs as a virtuous cycle in which more usage produces more data, which trains better algorithms, which deliver better service, which draws more usage (Iansiti and Lakhani, 2020). The point of the factory is not to do an old task cheaply; it is to generate predictions and decisions at a scale and consistency no human organisation could. In such a firm, the authors note, the machinery sits at the core of value delivery and “humans are moved to the edge, off the critical path” — a line to hold carefully, because where exactly the human belongs is the governance question this module spends the rest of its weeks on.

For your own company the practical lesson is this. When you argue for intelligent automation, do not argue only that it is a cheaper clerk. Argue what the shift from automating tasks to automating

decisions actually buys: decisions made consistently every time, at the full scale of the work rather than a sample of it, and the pattern-level insight that falls out when a machine reads everything. The cost saving is real and worth stating. But the strategic case — the one a board funds as an investment rather than an expense — is that the company starts to see and decide at a scale it could not before. That is the difference between a faster back office and a smarter company.

Rules-based automation sells on cost; intelligent automation sells on insight. A bot does an old task more cheaply. A model that reads all of the work — every contract, every message — makes decisions consistently at full scale and surfaces patterns no one had the hours to find. Iansiti and Lakhani call the mature version an AI factory: decision-making run as an industrial process. The board-level case for intelligent automation is not a cheaper clerk; it is a company that sees and decides at a scale it could not reach before.

Stop Pushing Work in Batches; Start Reacting to Events

There is a quieter shift underneath the move to intelligent automation, and if you miss it you will bolt clever models onto a dumb process and wonder why nothing improves. It is a shift in when a system acts. Most legacy processes are built on a batch-push rhythm: work is gathered up and pushed through on a schedule — the nightly run, the weekly reconciliation, the monthly close. Everything waits for the next scheduled push. That rhythm was a reasonable compromise when computing was expensive and integration was hard, and rules-based automation inherited it: a bot that runs the overnight batch is still living in the batch world, just faster. The trouble is that variance — the exception, the error, the case that needs attention — does not arrive on a schedule. It happens whenever it happens, and in a batch world it sits unseen until the next run picks it up, by which point the damage may be done.

Intelligent and agentic systems ask for a different rhythm: event-driven, where a trigger fires at the point of variance — the moment a case deviates — rather than waiting for the next batch. This is the process-philosophy shift the module turns on. A system that can read a situation in the moment is wasted if it is only allowed to look once a night; its whole advantage is that it can notice and respond when the thing actually happens. You can see the two modes already in the architecture Iansiti and Lakhani describe: a mature data platform carries both batch data and a stream of events and messages, so that applications can act on a flow of live signals, not just a nightly snapshot (Iansiti and Lakhani, 2020). And it is implicit in the way Tella and his co-authors describe an enterprise platform that watches for exception patterns and reacts to them as they emerge, rather than on a timetable (Tella, Brinker and Pezzini, 2023).

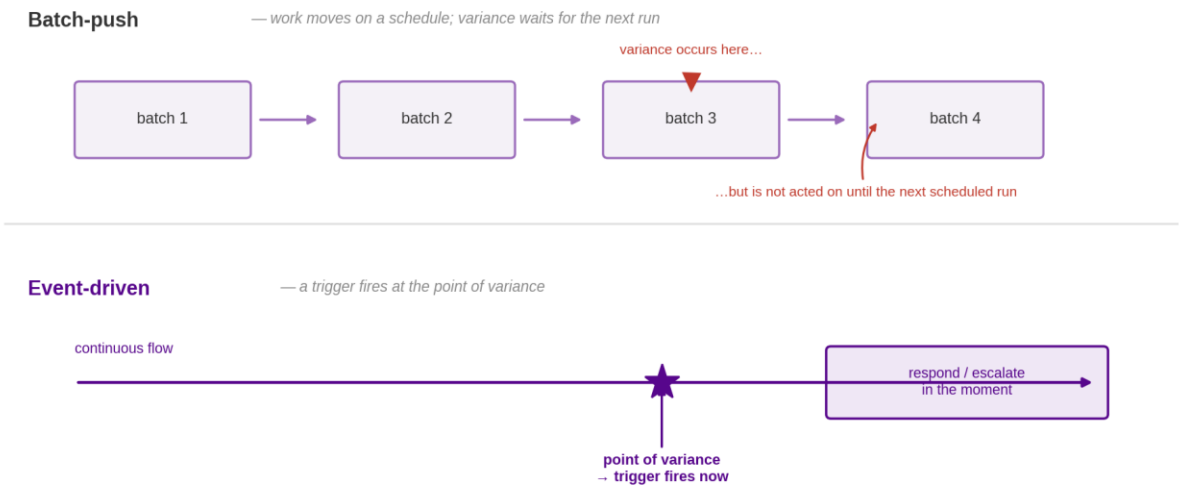


Figure 3. Batch-push moves work on a schedule, so a variance sits unseen until the next run. Event-driven fires a trigger at the point of variance and responds in the moment. Intelligent and agentic systems need the second rhythm; bolting them onto the first wastes their main advantage.

For your company this is a diagnosis to make, not just a concept to recite. Ask of the judgment step you are about to name: when does the organisation currently find out that this case needs attention — in the moment, or at the next scheduled run? If the answer is “at the next run,” then even a perfect model will be late, and the design has to change the rhythm, not just add intelligence. Without that shift, you will have paid for an intelligent system and handed it a batch-world job — a faster version of the same dysfunction. The rung you are climbing is not only smarter; it reacts to events, at the point of variance, which is where the value of seeing in the moment is actually captured.

Intelligence reacts to events; rules ran in batches. Legacy processes push work on a schedule, so variance waits for the next run. Intelligent and agentic systems need an event-driven rhythm — a trigger that fires at the point of variance and responds in the moment. A mature data platform carries both a batch store and a live event stream (Iansiti and Lakhani, 2020). Add intelligence without changing the rhythm and you get a faster version of the same dysfunction; the gain is captured only when the system can act when the thing actually happens.

The Platform That Makes AI Usable at Enterprise Scale

A model that reads contracts and a trigger that fires on an event are capabilities; they are not yet a company-wide capability. The gap between the two is the subject of Tella, Brinker and Pezzini's chapter on the enterprise AI platform, and it is worth taking seriously because it is the difference between a clever demo and something an organisation can actually run. Their starting observation is bracing and important: today's generative AI, for all its fluency, is “knowledgeable” but not reliably smart — these are “language models, not knowledge models,” producing statistical approximations of an answer rather than a verified one (Tella, Brinker and Pezzini, 2023). Left to

itself, a raw model will confidently produce output that is wrong, and no enterprise can run its operations on confident guesses. So the model needs a partner that supplies what it lacks, and that partner takes two forms the authors return to throughout: orchestration and governance.

Orchestration is the part that turns a model's answer into a real action inside your messy estate of existing systems. A raw model can draft an email or suggest a classification; it cannot, on its own, pull the record from NetSuite, update the ticket in your service tool, and notify the right person. An execution engine does that — it knits the model's output to every other system and tool the company already runs, so an idea becomes a completed piece of work. The platform also, the authors argue, has to meet the business where it is through three kinds of input: human-driven, where a person describes in plain language what they want automated; auto-discovery, where the platform mines the organisation's own processes to suggest where automation would pay — the long-promised idea of process intelligence, a system reading how work actually flows and pointing at the steps worth changing; and business-reactive, where it watches for exception patterns and proposes a response. Hold process intelligence lightly for now: it is one of the platform's input methods, the system's ability to look at your processes and tell you where the judgment steps and the waste are, rather than waiting for you to find them by hand.

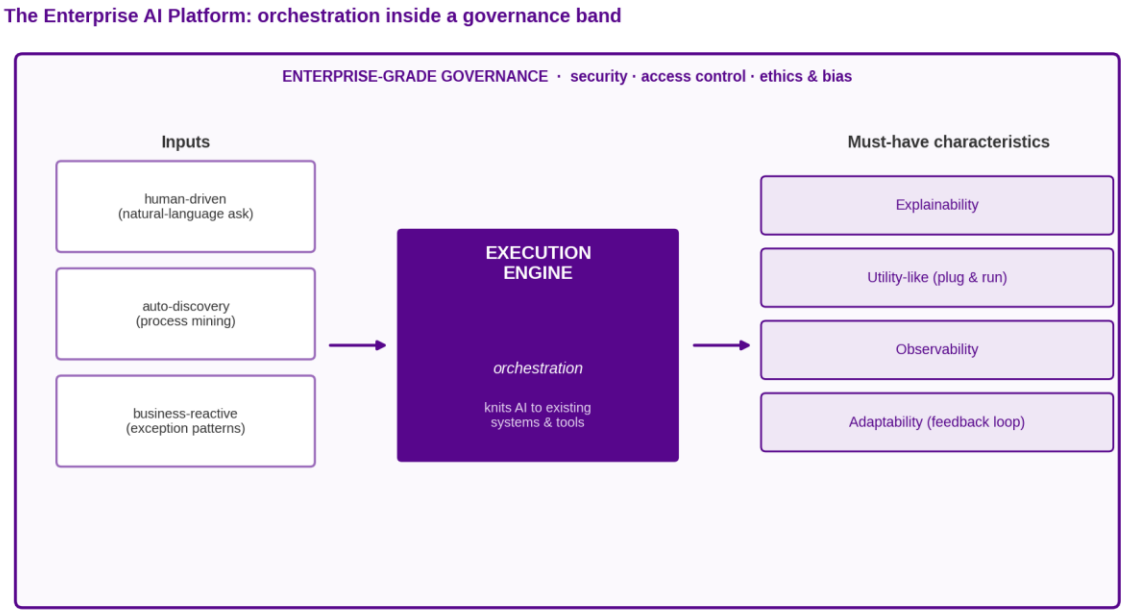


Figure 4. An enterprise AI platform surrounds a model with an execution engine that orchestrates existing systems, a set of must-have characteristics, and a governance band around the whole. Adapted from Tella, Brinker and Pezzini (2023), Ch. 15.

Governance is the other half, and it is where this week's ethics question starts to get its answer. Because the model is not reliably right, the authors insist on a set of characteristics that keep a human able to stay in control. Explainability means a solution is understandable to the non-technical person who has to validate it — if only an engineer can check the model's work, you have not democratised anything, you have just moved the bottleneck. Observability means someone can see what the system is doing and intervene when it goes wrong. Adaptability means the system learns from its mistakes through a feedback loop rather than repeating them. And

wrapped around all of it is enterprise-grade governance: access controls that say which processes a given user may automate and which systems a solution may touch, plus explicit handling of the new risks AI brings — ethics, bias, and the confident production of misinformation (Tella, Brinker and Pezzini, 2023). The reason these matter is the reason the whole module exists: the moment a system can act on its own reading of a situation, the question is no longer only "does it work?" but "who can see what it did, who can stop it, and who is answerable when it is wrong?"

Platform capability	What it does	Why a judgment-taking system needs it
Orchestration (execution engine)	Turns a model's output into real action across existing systems and tools	A reading or a decision is worthless until it moves the record, the ticket, the notification
Explainability	Makes a solution understandable to the non-technical person who must validate it	If only an engineer can check it, the human-in-the-loop cannot actually stay in the loop
Observability	Lets someone see what the system is doing and step in	A confidently-wrong model must be caught in the moment, not months later
Adaptability	Learns from errors through a feedback loop	The cases a rule could not foresee are exactly the ones the system must learn from
Enterprise governance	Access control over what may be automated and touched; handles bias, ethics, misinformation	When a machine takes an expert's step, control and accountability must be designed in, not assumed

A model is a capability; a platform is a governed capability. A raw model is fluent but not reliably right — "language models, not knowledge models" (Tella, Brinker and Pezzini, 2023). To run one at enterprise scale it needs two partners: orchestration, which turns its output into real action across your systems, and governance, which keeps a human able to see, validate, and stop it. Explainability, observability, adaptability, and access control are not extras; they are what make a judgment-taking system safe to deploy.

Why Agents, and Why Now

Everything so far has described intelligent automation — a model taking a judgment step inside a flow you designed. The rest of Module 3 is about something further along the ladder: the AI agent, a system that does not just take one step but pursues a goal, deciding for itself what steps to take. This note does not teach how an agent works — that is next week — but it does owe you the on-ramp, because the question a board will ask is not "how does it work?" but "why is this happening now?" Bornet and his co-authors give the clearest answer, and it is a story of two technologies finally meeting (Bornet et al., 2025).

For two decades, they argue, two separate streams ran in parallel. One was the rise of language models — the long climb from systems that could play chess but not explain a move, through neural networks, to the transformer breakthrough of 2017 and the large language models that followed. By 2022 these models could understand and reason in language with startling fluency — but, the authors note, they were “brilliant systems that can think but can't act.” They could suggest a decision; they could not reach into a company's systems and carry it out. The other stream was the evolution of automation — from brittle screen-scraping scripts, through RPA, to the intelligent automation this week describes. By the early 2020s automation had hit what the authors call the automation plateau: the straightforward, rules-based tasks were largely automated, and the work that remained was precisely the decision points and dynamic situations that still needed a person. Traditional automation could go no further, because going further meant judgment, and rules do not have judgment.

The agent is what you get when the two streams meet. The language model supplies the brain — the ability to understand a situation, reason about it, and plan; the automation layer supplies the hands — the ability to act in the company's real systems. Put them together and you have a system that can both decide what to do and do it. That is why now: not because anyone invented a single new thing, but because the thinking stream and the acting stream matured at the same moment and could finally be joined. On the ladder from Figure 1, this is the step from Level 2, where a model takes a judgment step inside a flow you control, to Level 3 and above, where an agent takes a goal and works out the flow itself.

Next week names the four capabilities an agent needs to do this — sense, plan, act, and reflect, which Bornet and his co-authors label the SPAR framework — and Lab 3 has you build one. For this week, carry only the framing, plus one discipline the authors are emphatic about, because it governs your homework. They call it the golden rule: the simpler, the better. The goal is never maximum autonomy; it is the right level of autonomy for the specific step and its risk. A bank runs its high-stakes, auditable transactions with low-level, predictable automation and reserves the adaptive, agentic kind for places where context and variation genuinely reward it. Read against your own map, that means: do not reach for an agent where a rule would do, and do not leave to a rule the one step that genuinely needs judgment. The whole of this week's homework is to find that one step and justify it — to say precisely where, on your process, a rule runs out and judgment begins.

A last word, because it is the thread the module never drops. Bornet and his co-authors are insistent that the agent is augmentative, not a replacement — best understood, they suggest, as a very capable new hire you would never let run unsupervised. In their own study of companies deploying these systems, the configuration that worked kept a human in the loop for the consequential decisions while the machine carried the routine (Bornet et al., 2025). That is the same human-in-the-loop principle you met in Week 5, now aimed at a more capable machine: the more a system can decide for itself, the more deliberately you have to choose where a person still must. Which is exactly the governance question the case now makes concrete.

Level	What it is	Where it sits in this course
Level 1 — Rule-based automation	Follows fixed rules; RPA and simple scripts; no judgment	Module 2 — what you built and broke in Lab 2
Level 2 — Intelligent automation	Adds ML / NLP to read unstructured input and make decisions	This week — the step you are taking now
Level 3 — Agentic workflows	Plans, reasons, and acts toward a goal within set bounds	Weeks 8–10 — SPAR, built in Lab 3
Levels 4–5 — Autonomous agents	Pursue goals across domains with little human input (largely still ahead)	The horizon the roadmap reckons with in Module 4

Agents are here because thinking and acting finally met. Language models gave machines a brain that could reason but "can't act"; automation gave them hands that could act but could not judge. The agent is the two joined — a system that decides what to do and does it (Bornet et al., 2025). The golden rule is the simpler, the better: match autonomy to the step's risk, keep a human in the loop for the consequential calls, and treat the agent as a capable new hire, not a replacement. Your homework is to find the one step on your map that genuinely needs that judgment.

The Case: A Bank Teaches a Machine to Read Its Own Contracts

Everything the week has argued is visible in one back-office story. In 2017 JPMorgan put into production a system it called COiN — short for Contract Intelligence — to review commercial loan agreements (Son, Bloomberg, 2017). Read it as the worked example of a machine taking a judgment step, and watch how cleanly it separates intelligent automation from the rule-based kind.

The problem. A large bank writes a great many commercial loan agreements — on the order of twelve thousand new ones a year — and every one has to be read and interpreted: what are its terms, its obligations, its exceptions? This is not clerk work. Interpreting a contract clause is a judgment step, the kind a rule cannot take, and the bank was spending roughly 360,000 lawyer- and loan-officer-hours a year doing it by hand (Son, Bloomberg, 2017). Worse, because interpretation is hard and the volume is high, human readers made mistakes, and those mistakes showed up as loan-servicing errors downstream. This is a judgment decision point at industrial scale: unstructured input (the contract text), an incomplete-by-definition rulebook (every contract is a little different), and a costly error pattern when it goes wrong.

What happened. COiN did not approach the contracts as a rule-based checker would — matching fixed keywords against a fixed template. It was built on machine learning: in the plain description the reporting carries, the system "learns by ingesting data to identify patterns and relationships," generalising from many examples to read agreements it had not seen before (Son, Bloomberg, 2017). Where a human reader took many minutes per contract, COiN reviewed documents in

seconds, and because it read consistently it reduced the servicing errors that human interpretation had been producing. The hours that had gone into reading moved elsewhere, and the bank began looking for other document-heavy judgment steps to point the technology at next.

The lesson — and it is the governance lesson, not the technical one. COiN is the exact contrast this week's third study question asks you to draw. A rules-based document checker matches patterns someone specified in advance and breaks the moment a contract is phrased in a way the template did not foresee. A machine-learning system learns to interpret, generalising to new phrasings the way a trained reader would. That is the difference between automating a clerk and automating a reader — and it is a difference in kind. When a machine takes over a step that genuinely required expert judgment, the questions change. If COiN misreads a clause, who is accountable — the model, the team that trained it, the officer who relied on it? What has to be transparent about how it reached its reading, so that a human can check it? And where, in a flow that now runs in seconds, does a person still have to stay in the loop — on the high-value agreement, the unusual clause, the case the model flags as uncertain? Notice that the bank did not propose to remove human judgment from lending; it moved the routine reading to the machine and left the consequential calls with people. That placement — what the machine takes and where the human stays — is not a detail. It is the design, and it is what your Board Memo will have to get right.

The thread. COiN lands first on Operations and on Cash Management — faster, more accurate contract review speeds the lending operation and tightens the cash it governs — which is why it is the right case to prove the mechanism even though Module 3 leads with the Customer thread. The move it demonstrates — a machine taking a judgment step that used to need an expert — is the same one that, aimed at a service or dispute-resolution step, will land squarely on the Customer thread in the agent weeks ahead.

COiN is a judgment step, taken by a machine, in the back office. A bank was spending 360,000 hours a year having people read and interpret its loan contracts, with errors when they got it wrong. COiN did not check the contracts against a rule; it learned to read them, and reviewed them in seconds with fewer servicing errors (Son, Bloomberg, 2017). That is intelligent automation, not RPA — a machine taking a step that needed expert judgment. The lesson is the governance one: when the reader is a model, you must design what it takes, what it must make transparent, and where a human still decides.

Where This Points: The Threads, the Labs, and the Capstone

Pull the week together on the three axes the course always uses. On the threads, this is the week Customer moves to the front of the module, even though the mechanism is proved on Operations and Cash. When you design for your own company, tag the value of the judgment step by thread — a faster, more accurate decision speeds cash, or removes a customer's wait, or steadies an operation — because a board funds a thread outcome, not a capability in the abstract.

On the labs, this week is the hinge between the two you have done and the one ahead. Lab 1 mapped your process and marked both kinds of decision point. Lab 2 built the stable-rules one

as a bot and broke it on the judgment one — giving you first-hand, in your own data, the experience of a rule running out. This week names that judgment step as the thing an intelligent system takes. Week 8 gives you the architecture of the system that takes it — the SPAR framework. And Lab 3, in Week 9, has you build an agent for that exact step on a no-code platform and then attack it with deliberately difficult inputs to find where it, too, fails. The spine is unbroken: the step you could not rule in Lab 2 is the step you will build for in Lab 3, and this week is where you pin down precisely what that step is.

On the capstone, your work this week opens the project's Design milestone (Milestone 2) and sets up Board Memo §5, Agent Design and Governance. The homework is deliberately narrow: on your own process, identify the one step that genuinely needs AI judgment — not three, not a wish-list, but the single clearest place where a rule runs out — and write why rules fail there. That one step, with its rationale, is the seed of §5 and the target your Lab 3 agent will take. It fills the first part of your agent-design slide; Week 8 adds the architecture, and Lab 3 adds the build and the guardrails. Keep the golden rule in view as you choose: the point is not to find the most impressive place to put an agent, but the one place where judgment is genuinely required and a rule genuinely cannot stand in. A board believes a design that is aimed precisely far more than one that reaches everywhere.

This week names the step; Lab 3 builds for it. Customer leads the module now; tag your judgment step's value by thread. The lab spine is unbroken — Lab 2 broke at the judgment point, Week 8 gives the architecture, Lab 3 builds the agent for it. Your homework opens the capstone's Design milestone and seeds Board Memo §5: name the one step on your process that genuinely needs AI judgment, and say why rules fail there. Aim precisely — one step, well justified, beats a wish-list.

What Comes Next — and What This Note Deliberately Leaves for Later

Two directions are handed forward on purpose; reach for them next week, not this one. Week 8 opens the agent itself: how it works, through the SPAR framework — sense, plan, act, reflect — and the capabilities beneath each part of that loop. The reading that prepares it is Bornet et al., Ch. 3, “Inside the Mind of an AI Agent”; read it, with Ch. 1–2 behind you, before the Week 8 session. Lab 3 then builds an agent in Week 9, and the deeper governance of autonomous systems — kill switches, accountability, the human-AI boundary at its hardest — is the work of Week 10 and the roadmap weeks of Module 4. The event-driven architecture this note sketched in concept is developed into real design later in the module and the roadmap; here you need only the shift in rhythm, not the engineering. Teach yourself none of this yet. This week's job is complete when you can say, of your own process, exactly where the rule runs out.

Study Questions

Bring reasoned answers; they are designed to be argued with the case and the frameworks, not recited.

1. **What types of tasks cannot be handled by rules-based RPA, and why does that boundary matter strategically?** Explain the two ways a rule runs out — unstructured input and the unforeseen case — in your own words, then say why knowing exactly where that boundary falls changes where a company should spend its next, more expensive layer of automation. Use the judgment decision point on your own Lab 1 map, or the COiN contracts, to make it concrete.
2. **How does the shift to intelligent automation change the value proposition from cost-saving to insight generation?** Contrast the business case for a rule-based bot (labour hours removed) with the case for a model that reads all of the work. What does a board get from the second that it does not get from the first, and why is that an investment rather than an expense?
3. **Using JPMorgan COiN, explain the difference between a rules-based document checker and a genuine ML-driven intelligence system.** Say what each does with a contract it has never seen before, why one generalises and the other breaks, and then name the governance change that follows when the reader is a model rather than a clerk — who is accountable, what must be transparent, and where a human stays.

Running Glossary

New this week, plus the carried terms this note builds on.

Intelligent automation. Rules-based automation augmented with cognitive abilities — machine learning, natural-language processing, computer vision — so it can interpret unstructured input and make decisions, not just execute fixed steps. Level 2 on the automation ladder (Bornet et al., 2025).

Judgment decision point. A decision point where the rule runs out — the cases vary, the input is unstructured, or the situation was not anticipated — so a person (or an intelligent system), not a rule, must weigh it. The target for intelligent automation and, in Lab 3, for an AI agent. (Contrast: stable-rules decision point, the RPA candidate from Week 4.)

Unstructured data. Information that carries meaning but does not sit in a labelled field a rule can read — free text, contract clauses, images, audio. Reading it requires machine learning, not a rule.

Machine learning (ML). A system that learns the pattern connecting inputs to outcomes from many examples, rather than following a rule written in advance, and applies it to cases it has not seen. Three families: supervised (reproduce a labelled expert judgment), unsupervised (find the natural grouping or the anomaly), and reinforcement (learn by trying toward a goal) (Iansiti and Lakhani, 2020).

Natural-language processing (NLP). The branch of machine learning that lets a system read and interpret human language — an email, a message, a document — so an automated flow can act on its meaning.

AI factory. Iansiti and Lakhani's term for a firm that runs decision-making as an industrial process: data pipelines, algorithms that generate predictions, an experimentation platform, and a

virtuous cycle in which more usage yields more data, better algorithms, and better service (Iansiti and Lakhani, 2020).

Batch-push vs. event-driven. Two rhythms for when a system acts. Batch-push moves work on a schedule, so variance waits for the next run. Event-driven fires a trigger at the point of variance and responds in the moment. Intelligent and agentic systems need the event-driven rhythm.

Point of variance. The moment a case deviates from the norm — the exception, the error, the thing that needs attention. An event-driven system acts here, rather than waiting for the next scheduled batch.

Enterprise AI platform. The layer that makes AI usable across a company by surrounding a model with orchestration (turning its output into real action across existing systems) and governance (keeping a human able to see, validate, and stop it) (Tella, Brinker and Pezzini, 2023).

Orchestration (platform sense). An execution engine that knits a model's output to the company's existing systems and tools so an idea becomes completed work. Distinct from the automation-tooling sense introduced in Week 5 and from the Week-11 roadmap phase of the same name — do not conflate the three.

Process intelligence. A platform's ability to mine how work actually flows and point at the steps worth changing — the judgment points and the waste — rather than waiting for a person to find them by hand. One of the enterprise platform's input methods (Tella, Brinker and Pezzini, 2023).

Explainability / observability / adaptability. Three platform characteristics that keep a human in control of a judgment-taking system: a solution a non-technical person can understand and validate; the ability to see what the system is doing and intervene; and a feedback loop through which it learns from its errors (Tella, Brinker and Pezzini, 2023).

Agentic AI. A system that pursues a goal by deciding for itself what steps to take, formed when a language model (the brain — reasoning and planning) is joined to an automation layer (the hands — action in real systems). Levels 3 and above on the ladder; its architecture (SPAR) is the subject of Week 8 (Bornet et al., 2025).

SPAR (previewed). Bornet and co-authors' four capabilities of an AI agent — sense, plan, act, reflect. Named here as the destination; its architecture is taught in Week 8, built in Lab 3.

Automation plateau. The point most organisations reach where the straightforward, rules-based tasks are automated and the remaining work is the decision points that need judgment — the ceiling rules-based automation cannot pass (Bornet et al., 2025).

Human-in-the-loop (carried, Week 5). A deliberate design choice to route certain cases to a person — ambiguous, high-value, or irreversible ones. With an intelligent or agentic system, the more the machine can decide, the more deliberately the human's place must be designed.

Faster versions of the same dysfunction (carried). What a company gets when it adds speed or intelligence to a broken process without changing the process philosophy — automating a judgment step with more rules, or adding a model to a batch-world job.